

Terraform - Cheat Sheet

Contents

1. [The workflow](#)
2. [HCL essentials](#)
3. [Providers](#)
4. [Resources](#)
5. [Data sources](#)
6. [Variables](#)
7. [Outputs](#)
8. [State](#)
9. [Modules](#)
10. [Workspaces](#)
11. [Lifecycle and meta-arguments](#)
12. [Dynamic blocks](#)
13. [Functions](#)
14. [Targeted operations and replacement](#)
15. [Importing existing resources](#)
16. [Common patterns](#)
17. [Terragrunt \(brief\)](#)
18. [Best practices](#)
19. [Common gotchas](#)
20. [Quick reference](#)

Terraform is HashiCorp's infrastructure-as-code tool. You write declarative `.tf` files describing what cloud resources you want, run `terraform apply`, and Terraform figures out how to make reality match. It tracks what it created in a state file and only changes the resources that drifted.

Works across AWS, GCP, Azure, Kubernetes, Datadog, GitHub, and hundreds more providers. Same syntax everywhere. The trade-off vs cloud-native tools (CloudFormation, Deployment Manager) is portability for slightly more setup.

1. The workflow

Four commands cover 95% of daily use.

```
terraform init      # download providers, initialize backend
terraform plan      # show what would change
terraform apply     # make it happen
terraform destroy   # tear it all down
```

`plan` is read-only. Always run it before `apply` so you know exactly what's going to change.

```
terraform fmt      # format files
terraform validate  # syntax check without hitting providers
terraform show      # current state
terraform state list # list resources in state
```

2. HCL essentials

HCL (HashiCorp Configuration Language) is the syntax. It's declarative, with some imperative-looking constructs.

```
# A block: resource type + label + body
resource "aws_instance" "web" {
    ami           = "ami-0c55b159cbfafa1f0"
    instance_type = "t3.micro"

    tags = {
        Name = "web-server"
        Env  = "prod"
    }
}

# Variable
variable "region" {
    type     = string
    default = "us-east-1"
}

# Reference: <resource_type>.<name>.<attribute>
output "web_ip" {
    value = aws_instance.web.public_ip
}
```

Types: `string`, `number`, `bool`, `list(T)`, `set(T)`, `map(T)`, `object({})`, `tuple([])`, `any`.

Interpolation:

```
name = "web-${var.environment}-${count.index}"
```

3. Providers

Providers are plugins that talk to cloud APIs. Declare them at the top:

```
terraform {
  required_version = ">= 1.5.0"

  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
    }
    google = {
      source = "hashicorp/google"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}

provider "google" {
  project = "my-project"
  region = "us-central1"
}
```

Pin versions. `~> 5.0` means “5.x but not 6.0”. Without a lock, upgrades can break your plans.

Multiple instances of the same provider via aliases:

```
provider "aws" {
  region = "us-east-1"
  alias = "east"
}

provider "aws" {
  region = "eu-west-1"
  alias = "europe"
}

resource "aws_s3_bucket" "logs_eu" {
  provider = aws.europe
  bucket   = "my-logs-eu"
}
```

4. Resources

A resource is something Terraform creates and manages.

```

resource "aws_s3_bucket" "data" {
  bucket = "my-app-data"

  tags = {
    Owner = "team-platform"
  }
}

resource "aws_s3_bucket_versioning" "data" {
  bucket = aws_s3_bucket.data.id

  versioning_configuration {
    status = "Enabled"
  }
}

```

The first label is the resource type (`aws_s3_bucket`). The second is the local name (`data`). Other resources reference this one as `aws_s3_bucket.data.<attribute>` .

5. Data sources

Read-only queries to look up existing resources or external info.

```

data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"]

  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-*"]
  }
}

resource "aws_instance" "web" {
  ami           = data.aws_ami.ubuntu.id
  instance_type = "t3.micro"
}

```

Useful for looking up account IDs, VPC IDs, AMIs, existing IAM roles, secrets in a vault.

6. Variables

Three places to define inputs.

Variable declarations

```
variable "environment" {
  type      = string
  description = "Deployment environment"
  default   = "dev"

  validation {
    condition     = contains(["dev", "staging", "prod"], var.environment)
    error_message = "environment must be dev, staging, or prod."
  }
}

variable "db_password" {
  type      = string
  sensitive = true
}

variable "tags" {
  type    = map(string)
  default = {}
}
```

Values

In priority order (later wins):

1. Environment variables: `TF_VAR_environment=prod`.
2. `.tfvars` files: `terraform apply -var-file=prod.tfvars`.
3. Command-line flags: `-var "environment=prod"`.
4. Automatic loading: `terraform.tfvars`, `*.auto.tfvars`.

`sensitive = true` keeps the value out of plan and apply output (still in state).

7. Outputs

Surface values that other tools or modules need.

```
output "vpc_id" {
  value      = aws_vpc.main.id
  description = "Main VPC ID"
}

output "db_password" {
  value      = aws_db_instance.main.password
  sensitive  = true
}
```

After `terraform apply`, outputs print and can be read with `terraform output vpc_id`.

8. State

The state file (`terraform.tfstate`) is Terraform's map between config and the real world. It tracks resource IDs, attributes, dependencies.

Why state matters

- Without state, Terraform can't tell what already exists.
- State holds sensitive values (DB passwords, keys). Treat it like a secret.
- Two engineers running Terraform against the same target need shared state or they'll fight.

Remote state

Local state files don't share. Production should always use remote state.

```
terraform {
  backend "s3" {
    bucket      = "my-tf-state"
    key         = "prod/network.tfstate"
    region      = "us-east-1"
    encrypt     = true
    dynamodb_table = "terraform-locks"
  }
}
```

Backends:

- `s3` (with DynamoDB for locking)
- `gcs` (Google Cloud Storage)
- `azurerm`
- `remote` (Terraform Cloud / Enterprise)
- `http` (custom)

State locking

Two people running apply at the same time corrupts state. Locking blocks the second one.

- S3 backend uses DynamoDB for locks.
- GCS has native locking.
- Terraform Cloud locks automatically.

Force-unlock only if you're sure no one is running:

```
terraform force-unlock <LOCK_ID>
```

9. Modules

A module is a reusable bundle of resources. Anything you do once you might do again.

Module structure

```
modules/vpc/  
  main.tf          # resources  
  variables.tf     # inputs  
  outputs.tf       # outputs  
  versions.tf      # provider requirements
```

Calling a module

```
module "vpc" {  
  source = "../modules/vpc"  
  
  name      = "prod-vpc"  
  cidr_block = "10.0.0.0/16"  
  az_count  = 3  
}  
  
output "vpc_id" {  
  value = module.vpc.vpc_id  
}
```

Module sources

```
# Local  
source = "../modules/vpc"  
  
# Git  
source = "git::https://github.com/example/modules.git//vpc?ref=v1.2.0"  
  
# Terraform Registry  
source = "terraform-aws-modules/vpc/aws"  
version = "5.0.0"  
  
# S3  
source = "s3::https://s3.amazonaws.com/my-bucket/vpc.zip"
```

Pin versions when sourcing from registries or git. Without a pin, the module can change under you.

10. Workspaces

Multiple states for the same configuration. Useful for environments.

```
terraform workspace new staging
terraform workspace select prod
terraform workspace list
```

```
resource "aws_s3_bucket" "data" {
  bucket = "myapp-data-${terraform.workspace}"
}
```

Workspaces are stored separately in the backend. They share the same code but get their own state.

The community is split on workspaces vs separate directories per environment. Workspaces are simpler but can hide environment-specific changes. Many teams use directory-per-environment instead.

11. Lifecycle and meta-arguments

Every resource accepts these special arguments.

count and for_each

```
# count: make N copies
resource "aws_instance" "web" {
  count      = 3
  ami       = data.aws_ami.ubuntu.id
  instance_type = "t3.micro"

  tags = { Name = "web-${count.index}" }
}

# for_each: one per key in a map or set
resource "aws_iam_user" "developers" {
  for_each = toset(["alice", "bob", "charlie"])

  name = each.key
}
```

`for_each` is generally safer than `count` because removing an item doesn't shift indices and cause Terraform to recreate everything that comes after.

depends_on

Explicit dependency when Terraform can't figure it out from references.

```
resource "aws_iam_role_policy" "lambda" {
  # ...
  depends_on = [aws_iam_role.lambda]
}
```

lifecycle block

```
resource "aws_s3_bucket" "important" {
  bucket = "critical-data"

  lifecycle {
    prevent_destroy      = true
    create_before_destroy = true
    ignore_changes       = [tags]
  }
}
```

- `prevent_destroy`: refuses to delete this resource. Plan fails if it would.
- `create_before_destroy`: when replacing, create the new one first. Useful for downtime-sensitive resources.
- `ignore_changes`: don't react to drift on these attributes. Common for tags managed by other tools.

provider

Pick a non-default provider alias (see Providers section).

12. Dynamic blocks

Generate nested blocks programmatically.

```
resource "aws_security_group" "web" {
  name = "web"

  dynamic "ingress" {
    for_each = var.allowed_ports

    content {
      from_port = ingress.value
      to_port   = ingress.value
      protocol  = "tcp"
      cidr_blocks = ["0.0.0.0/0"]
    }
  }
}
```

If `var.allowed_ports = [80, 443]`, this produces two ingress blocks. Useful when the block count depends on input.

13. Functions

HCL has built-in functions for strings, collections, encoding, filesystem.

```
upper("hello")           # -> "HELLO"
length([1, 2, 3])        # -> 3
concat([1, 2], [3, 4])   # -> [1, 2, 3, 4]
merge({a = 1}, {b = 2})  # -> {a = 1, b = 2}
lookup(map, "key", "default")
jsonencode({foo = "bar"}) # -> "{\"foo\":\"bar\"}"
file("${path.module}/script.sh")
templatefile("config.tpl", {name = "x"})
formatdate("YYYY-MM-DD", timestamp())
```

There are no user-defined functions. If the built-ins don't have what you need, pre-compute in a script and pass via variables.

14. Targeted operations and replacement

-target

Limit a plan or apply to specific resources.

```
terraform plan -target=aws_instance.web
terraform apply -target=module.vpc
```

Use sparingly. Targeted applies skip dependency analysis and can leave state in odd shapes.

-replace

Mark a resource for replacement on the next apply.

```
terraform apply -replace=aws_instance.web
```

Replaces the older `terraform taint` workflow. Useful when a resource is in a bad state and you want to recreate it.

15. Importing existing resources

If a resource exists outside Terraform, import it into state.

```
import {
  to = aws_s3_bucket.existing
  id = "my-existing-bucket"
}

resource "aws_s3_bucket" "existing" {
  bucket = "my-existing-bucket"
}
```

(Terraform 1.5+ syntax.) Older versions used the CLI: `terraform import aws_s3_bucket.existing my-existing-bucket`.

After import, run `terraform plan` to verify the config matches the real resource. Edit the config until plan shows no changes.

16. Common patterns

Multi-environment with directories

```
environments/
  dev/
    main.tf
    backend.tf
    terraform.tfvars
  staging/
  prod/
modules/
  vpc/
  rds/
  eks/
```

Each environment has its own state and inputs. Modules are reused.

Layered state (per concern)

Split state by lifecycle:

- Network (VPC, subnets): rarely changes.
- Data layer (RDS, S3): changes occasionally.
- Compute layer (EKS, Lambda, ECS): changes often.

Each layer has its own state. The compute layer reads from network state via `terraform_remote_state` data source.

```

data "terraform_remote_state" "network" {
  backend = "s3"
  config = {
    bucket = "my-tf-state"
    key    = "prod/network.tfstate"
    region = "us-east-1"
  }
}

resource "aws_instance" "web" {
  subnet_id = data.terraform_remote_state.network.outputs.private_subnet_ids[0]
}

```

17. Terragrunt (brief)

A thin wrapper around Terraform. Adds DRY config, automatic remote state setup, and dependency management between modules.

```

# terragrunt.hcl
include "root" {
  path = find_in_parent_folders()
}

terraform {
  source = "../.. /modules/vpc"
}

inputs = {
  cidr_block = "10.0.0.0/16"
  name      = "prod-vpc"
}

```

Popular for managing many small Terraform modules across environments. Skip it if Terraform alone is enough.

18. Best practices

- Pin Terraform version (`required_version`) and provider versions (`~> X.Y`).
- Store state remotely with locking. S3 + DynamoDB or Terraform Cloud.
- Encrypt state at rest. Sensitive values land in state plain-text.
- Use modules for anything you build more than once.
- Split state by lifecycle. Network, data, compute as separate states.
- Run `terraform fmt` in CI. Inconsistent formatting causes pointless diffs.
- Always run plan before apply and read the output.
- Tag everything with `Environment` , `Team` , `Project` . Filters in the cloud console depend on it.

- Use `for_each` when iterating over named things. `count`-based lists shift indices on removal and trigger needless recreates.
- Keep secrets out of `.tf` files and `.tfvars` checked into git. Use a secret manager or environment variables.
- Use `terraform_remote_state` to share outputs between layers, sparingly. Tight coupling between states grows brittle.
- Review every `prevent_destroy` removal. That flag exists for a reason.

19. Common gotchas

- State drift. Someone edited a resource in the cloud console. The next plan shows changes you don't want. Either pull the changes into config (`import` or rewrite) or fix the resource to match config.
- Apply without plan. Recipe for surprise destruction. Always plan first.
- State file in git. Sensitive data leaks. State files don't belong in version control.
- Provider version drift. A `terraform init` on a fresh machine picks up a new minor version that introduces breaking changes. Pin with `~>`.
- Cycle in dependencies. Two resources referencing each other directly. Refactor to break the cycle.
- Forgetting `depends_on`. Implicit dependencies through references usually work. Sometimes (especially IAM and resource policies) Terraform creates things in parallel and one fails because the other isn't ready yet.
- Re-creating large resources accidentally. A change to a forces-new-resource attribute (like AMI ID on an instance) triggers replacement. Read the plan output carefully.
- `count` removal shifting indices. Removing element 0 in a `count`-based list shifts everything down and triggers a wave of re-creates. Use `for_each` with stable keys.
- State lock stuck. A previous apply crashed without releasing the lock. Wait for the TTL, or force-unlock if you're sure nothing's running.
- `terraform destroy` is too easy. It deletes everything in the state. Use `-target` or remove the lifecycle protections only when truly needed.
- Provider auth surprises. A workspace switch can mean different AWS credentials. Verify with `aws sts get-caller-identity` before applying to prod.
- Module versions in registry. A registry module with no version pin can update silently. Always pin.

20. Quick reference

Command	What it does
<code>terraform init</code>	Download providers, initialize backend
<code>terraform plan</code>	Show what would change
<code>terraform apply</code>	Make changes
<code>terraform destroy</code>	Delete everything in state

Command	What it does
<code>terraform fmt</code>	Format files
<code>terraform validate</code>	Syntax and type check
<code>terraform show</code>	Inspect state
<code>terraform state list</code>	List resources in state
<code>terraform state mv</code>	Rename a resource in state
<code>terraform state rm</code>	Remove from state (without destroying)
<code>terraform import</code>	Bring an existing resource into state
<code>terraform output</code>	Show output values
<code>terraform workspace</code>	Manage workspaces
<code>terraform plan -target</code>	Plan one resource only
<code>terraform apply -replace</code>	Force replace a resource

Concept	Key fact
Provider	Plugin for a cloud or service
Resource	A thing Terraform creates and manages
Data source	Read-only lookup
Variable	Input to a configuration
Output	Surface a value to callers
State	What Terraform thinks exists
Backend	Where state is stored
Lock	Prevents concurrent apply
Module	Reusable bundle
Workspace	Multiple states for the same code
lifecycle	Per-resource creation rules
for_each	One resource per key in a map or set
dynamic	Programmatic nested blocks